

Watching a local model think while it runs Hermes Agent

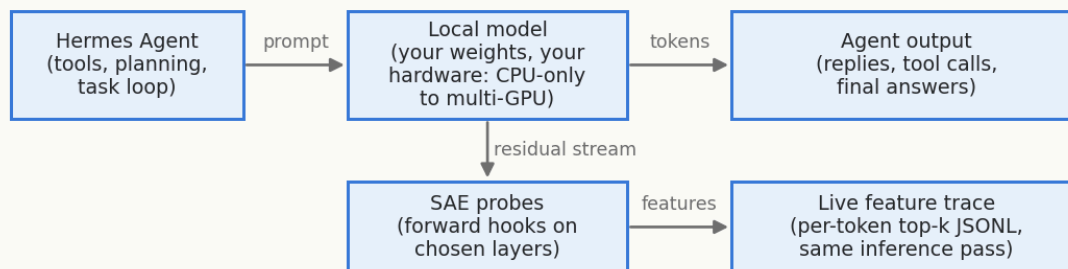
Caleb DeLeeuw ([SolshineCode](#)) — for Tristan and the Nous Research team, 2026-07-21.
Code and evidence: github.com/SolshineCode/hermes-sae.

What this is

While Hermes Agent runs on a local model, I attach sparse autoencoder (SAE) probes to the model's residual stream and log which learned features fire, per token, as the agent works. An SAE here is a learned dictionary that unmixes the model's dense internal activity into individual, human-inspectable features, the approach behind Anthropic's [Towards Monosemanticity](#) and [Scaling Monosemanticity](#) work. The transcript tells you what the agent said. The feature trace tells you what the model was doing internally while it said it: what lit up during planning, and what was active in the exact forward pass where a tool call went sideways.

The property that makes this trustworthy is what I call the same-inference invariant: the feature activations come from the *same* `model.generate()` call that produced the agent's output. No replay, no second hooked model whose activations might diverge from the ones that actually drove the behavior. One forward pass, two outputs: the tokens, and a readout of the internals that produced them.

One forward pass, two outputs: what the agent said, and what it was thinking



This only works because the weights are local. You can't hook the residual stream of an API: a closed model is a black box by construction, and local weights are a glass house the moment you bring instruments. The interpretability story and the local-model story are the same story, and that is why I'm bringing it to Nous: an agent stack where "what is my model actually doing" is a first-class, inspectable question feels like a natural

extension of what Hermes already is. Goodfire built roughly this as a hosted product (Ember) and then closed it to partners-only earlier this year. Nobody currently offers it in the form your users would actually want: self-hosted and open, wired into the agent loop.

Closing that gap is the point of this project. My mission with this work is to democratize access to mechanistic interpretability, the research side and the applied side both. Right now, watching a model's internals while it does real work is something you get at a frontier lab or through an enterprise product, and mostly nowhere else. The pieces to change that are already open: weights are local, good pretrained SAE dictionaries are sitting on HuggingFace, and Hermes is built to run local models. Wire them together and anyone with their own hardware can see what their model was doing on every agent turn, debug looping and drift from internal state instead of transcript guesswork, and do real interpretability work without anyone's permission. The field gets more trustworthy when far more people can practice it, not just read about it.

What runs today

Three tiers of evidence, from most Hermes-relevant to most statistical. All code, captures, and reports are in the repo.

1. A local model serving as the Hermes Agent model, with live capture. I ran Gemma-4-E2B (4-bit, on a 4 GB laptop GPU) as the acting model for a real Hermes Agent install, through an OpenAI-compatible server that hooks the residual stream inside its own `generate()` call. Hermes points at it with a stock custom-provider profile: `model.provider custom`, `model.base_url http://127.0.0.1:8000/v1`. No fork, no patch to Hermes. I picked a model I'd already built a dictionary for: the SAE for Gemma-4-E2B is one I trained and published on Hugging Face this spring ([gemma-4-e2b-scope-v1](#)). Every agent turn produces both the reply Hermes sees and a capture of the residual-stream activations of that exact inference, correlated by request. The recipe is [here](#).

That run also produced the first concrete diagnostic finding, and it cut in a direction I didn't expect. A 2B model is a weak agent, and this one degraded into repetition loops. The naive story would be "the internals collapse when the text loops." The data says something more interesting: on a turn where the text looped outright (the same token 96 times), the layer-23 residual stream did *not* reach a fixed point (no consecutive-token pair above 0.9 cosine), while across all fifteen captured turns the trajectory sat in a soft low-diversity attractor (consecutive-token cosine means of 0.46 to 0.67, individual pairs up to 0.93) that the transcript cannot see at all. A transcript monitor and an activation monitor disagree in both directions, which is exactly the argument for capturing both. Drift and degradation are measurable internal states, and they are not the same states the text shows you. Per-turn numbers are in the capture folder's [analysis](#).

2. SAE feature separability at 27B scale. On a bigger rig (Qwen3.5-27B in fp16 across two used Tesla M40s, under \$200 of GPU), I ran multi-step agentic labeling workloads through the same hooked-serving path, with SAE probes at five layers (dictionary width 81,920 per layer), and tested whether the features carry information about what kind of situation the model is in. On a 181-turn trace split between two scenario pools, 37 to 44 percent of features that fire in at least 20 turns survive Benjamini-Hochberg FDR at $\alpha=0.05$ as discriminators between the pools, at every one of the five layers. The strongest signatures are presence-only: features that fire on every turn of one condition and on zero turns of the other. Full statistics, the bugs found along the way, and exactly what this does and does not establish are in the [B1 report](#).

None of this has to be consumed as tables, either. For research-grade captures, the repo ships a static [dashboard](#) that renders a full-dictionary capture (top features per layer, a token-by-feature activation heatmap across all 81,920 entries) in any browser with nothing behind it. And for everyday use, the plugin carries its own session dashboard, described below.

3. Anyone can replicate the pipeline on a CPU in one command. The hardware above is what I happened to have, not what's required: the probe runs wherever the model runs, from CPU-only up. The same capture path runs on gpt2-small and Qwen2.5-0.5B with publicly released SAEs, no GPU: `./setup_cpu_test.sh qwen25b`. The same-inference invariant is verified end to end on both the 27B GPU path and the CPU path, including byte-identical generation text between engine-direct and HTTP-server modes.

One deliberate point about rigor: an earlier version of my statistics pipeline had real flaws, and I had it adversarially audited before trusting it. The [audit](#) is public in the repo, and the current analyzers are the post-audit redesign.

The Hermes Agent plugin

Alongside this memo I've prepared `sae_trace`, an observability plugin for Hermes Agent that follows the same read-only observer contract as the bundled Langfuse plugin. It correlates each agent turn with the feature-trace records the SAE-hooked server emits, writes a per-session interpretability sidecar, and documents the JSONL schema as an interface so any OpenAI-compatible server that emits it works, not just mine. It's built to run either as a standalone plugin in `~/.hermes/plugins/` (that copy lives in [hermes-plugin/sae_trace/](#)) or bundled under `plugins/observability/` with a 35-test suite. The bundled version is submitted as [PR #68543](#); whichever adoption path the team prefers works for me.

The plugin also gives Hermes users a watch-your-session view with zero install: it ships a single self-contained `dashboard.html` (no server, no build step, no network) that renders

the session's turns with their match confidence, the top SAE features per layer with activation bars, and a session-level feature aggregate. Pick the trace file once and the page follows it live as the agent works. `/sae dashboard` in any Hermes session prints where everything is.

The design principle throughout: Hermes stays untouched. The model side is just a serving wrapper, the agent side is just an observer plugin, and the contract between them is a documented sidecar schema.

How this relates to Nous's own interpretability work

I've read Nightwing's [neuron-steering post](#) carefully, and I want to engage its argument directly rather than talk past it. Contrastive Neuron Attribution deliberately avoids SAEs because training them is expensive and noisy, and for *intervention* at high steering strengths that tradeoff makes sense. This project sits on the other side of the same coin, and it dodges the training-cost objection: it consumes pretrained open SAEs rather than requiring anyone to train new ones. Goodfire's open-sourced SAEs for [Llama-3.1-8B-Instruct](#) and [Llama-3.3-70B-Instruct](#) are directly loadable by this probe, and since the Hermes model line shares the Llama base family, an obvious experiment is whether those dictionaries transfer usefully to Hermes checkpoints under fine-tuning. If they do, Hermes users get named-feature internals for their own models with zero training cost. If they don't, that's a publishable negative result about SAE transfer, found on consumer hardware.

The two approaches also compose in the direction I care about most. My roadmap ends at steering: catch a bad internal state live, looping or off-task drift, and nudge the model back on task mid-run. CNA is a credible steering backend for exactly that closed loop, with the SAE trace as the trigger and CNA neuron sets as the actuator. Read with one method, act with the other. I'd genuinely like to explore that with whoever did the [neural-steering](#) work.

What's established and what isn't

Established: same-inference capture at 27B scale and on CPU; a local model serving a real Hermes Agent install with per-turn capture; SAE features separating scenario conditions at FDR 0.05 across all probed layers; model-, SAE-format-, and hardware-agnostic pipeline.

Not established, stated plainly: whether the separability signal reflects the specific behavior under study or broader contextual differences between the scenario pools (the matched-pair test that disentangles this is pre-registered and next in queue); probe overhead in tokens per second (measurement scheduled, and it belongs in the plugin

README before anyone depends on it); any steering result at all, at any scale; SAE dictionary transfer to Hermes checkpoints. Feature labeling coverage is minimal so far. A feature trace is evidence about what a model is doing, not proof, and I won't present it as a safety guarantee.

What I'm offering

Everything below is built so a Hermes user can pick it up without me in the loop. Nothing depends on my hardware or my server.

1. **The repo**, MIT-licensed: serving layer, probe, analyzers, reports, and a one-command CPU replication.
2. **The `sae_trace` plugin**, PR-ready for the observability family or publishable standalone, whichever fits the team's preference.
3. **A demo on a Hermes model**. Hermes 4 plus an open Llama SAE under Hermes Agent is the experiment I most want to run next, and I'd shape it around whatever the team would find most useful.
4. **Collaboration on failure modes**. The Nous team knows better than anyone what actually goes wrong in long Hermes runs. A shortlist of "behaviors you wish you could see coming" would directly pick which features I label and track first, and would define the acceptance tests for the eventual steering loop.

Powerful AI in the hands of the many should include the ability to see inside it. Hermes already gives people sovereignty over their weights and their agent. This adds the missing layer: knowing what those weights are doing while the agent runs, on your own machine, answerable to no one. That capability shouldn't live behind an enterprise contract, and after this week it doesn't have to. I'd love the team's read on where it would be most useful.

Repo: github.com/SolshineCode/hermes-sae. I'm on Discord and happy to walk through any of it live.